Debian Server Setup

Running the Flask Web Application

Purpose:

The Debian virtual machine was used to host the target Flask web application during the simulation.

Observation:

The application was started successfully from the terminal, and the service was running on the assigned port, making the website accessible for monitoring and testing.

```
jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
(venv) debian@debian:~/Desktop/mcp-server$ nano app.py
(venv) debian@debian:~/Desktop/mcp-server$ sudo touch /var/log/flask_api.log
sudo chmod 666 /var/log/flask_api.log
[sudo] password for debian:
(venv) debian@debian:~/Desktop/mcp-server$ sudo ufw allow 5000/tcp
sudo ufw allow ssh
sudo ufw enable
sudo ufw status
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Firewall is active and enabled on system startup
Status: active

To                         Action      From
--                         ------      ----
5000/tcp                   ALLOW       Anywhere
22/tcp                     ALLOW       Anywhere
5000/tcp (v6)              ALLOW       Anywhere (v6)
22/tcp (v6)                ALLOW       Anywhere (v6)

(venv) debian@debian:~/Desktop/mcp-server$ python3 app.py
```

Open

app.py
~/Desktop/mcp-server

```python
from flask import Flask, request
import os

app = Flask(__name__)

@app.route('/')
def home():
    return "MCP Target Server Running"

@app.route('/run', methods=['POST'])
def run():
    cmd = request.json.get("cmd")
    output = os.popen(cmd).read()
    return {
        "command": cmd,
        "output": output
    }

@app.route('/login', methods=['POST'])
def login():
    username = request.json.get("username")
    password = request.json.get("password")

    if username == "admin" and password == "admin":
        return {"status": "success"}

    return {"status": "failed"}

app.run(host='0.0.0.0', port=5000)
```

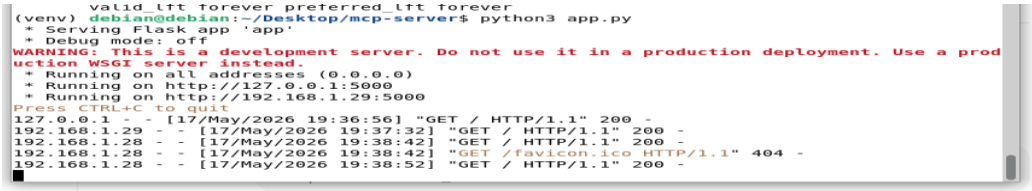
Running the Application

Command:

python3 app.py

Purpose:
Start the Flask web application.

Observation:
The application started successfully and began listening for incoming requests.



Creating the Flask Log File

Purpose:
Prepare application logging for monitoring.

Observation:
A dedicated log file was created to store Flask application events for Splunk analysis.

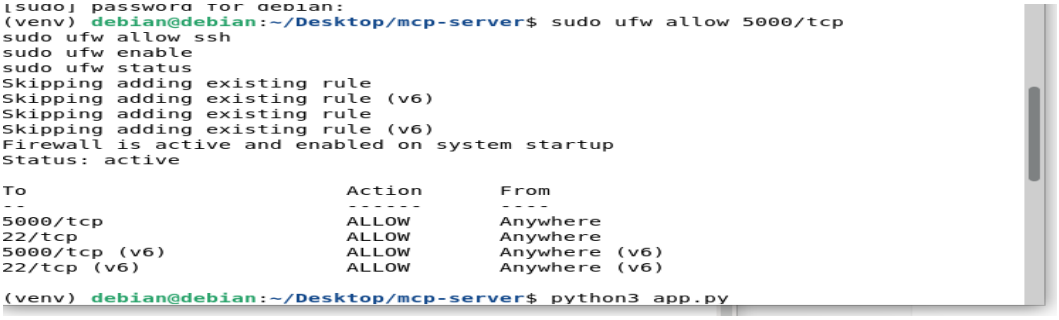
```
 * Debug mode: off
^C(venv) debian@debian:~/Desktop/mcp-server$ nano app.py
(venv) debian@debian:~/Desktop/mcp-server$ sudo touch /var/log/flask_api.log
sudo chmod 666 /var/log/flask_api.log
python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
```

Firewall Configuration

Command: sudo ufw allow 5000/tcp

Purpose:
Allow external access to the Flask application.

Observation:
Firewall rules were updated successfully and port 5000 became accessible.

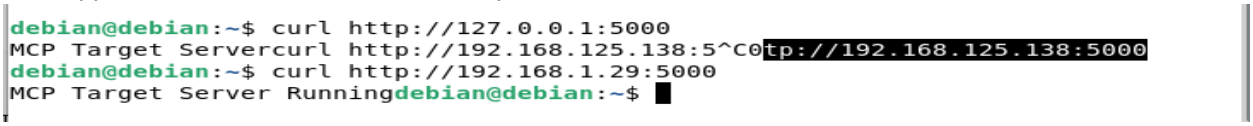


Localhost Access Verification

Command: curl http://127.0.0.1:5000

Purpose:
Verify that the application responds locally.

Observation:
The application returned a successful response on localhost.

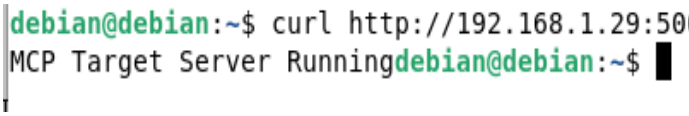


Network Access Verification

Command: curl http://192.168.1.29:5000

Purpose:
Confirm access from the network IP.

Observation:
The application was reachable through the assigned Debian IP.

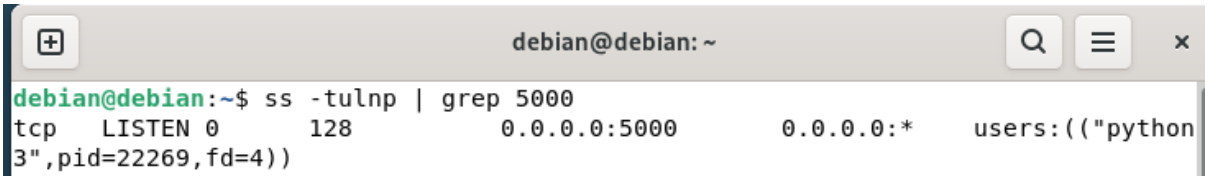


Checking Listening Port

Command: ss -tulnp | grep 5000

Purpose:
Verify that the Flask service is listening on port 5000.

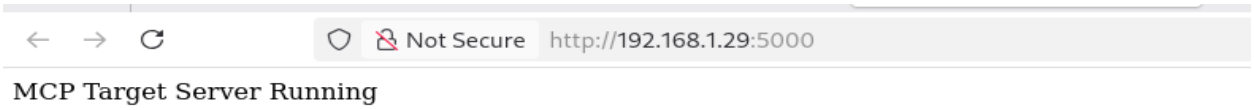
Observation:
The application was actively listening and ready for incoming connections.



Website Availability

Purpose:
Verify that the web application is accessible through the browser.

Observation:
The target website loaded successfully and displayed the application page.



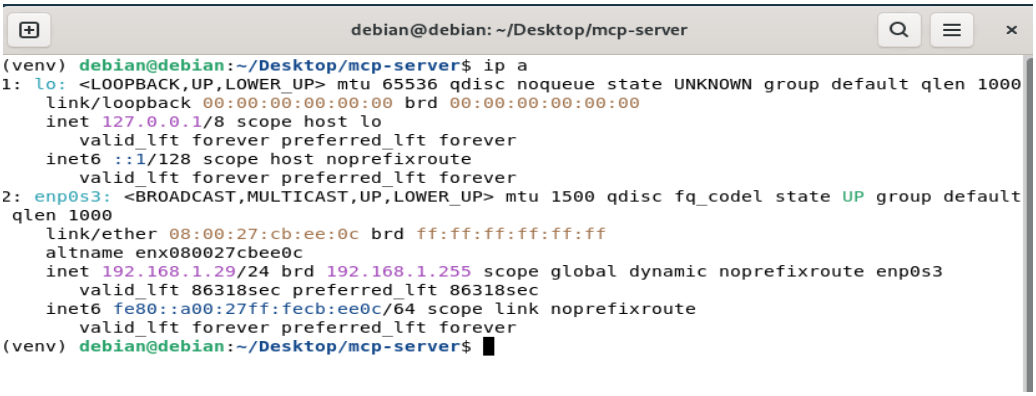
Debian Network & SSH Configuration

1 Checking Debian IP Address

Command: ip a

Purpose:
Identify the Debian machine IP address.

Observation:
The assigned address was confirmed as `192.168.1.29`



2 Enabling SSH Access

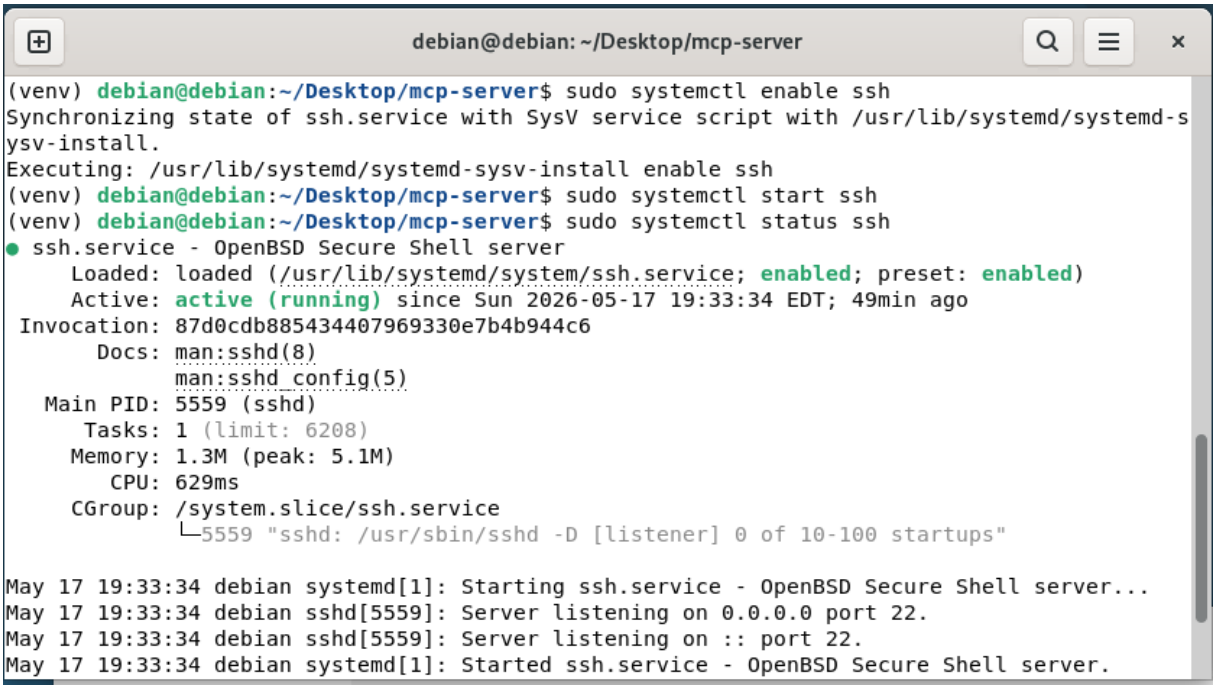
Commands: sudo systemctl enable ssh

sudo systemctl start ssh

sudo systemctl status ssh

Purpose:
Enable secure remote access.

Observation:
SSH was enabled and running successfully.

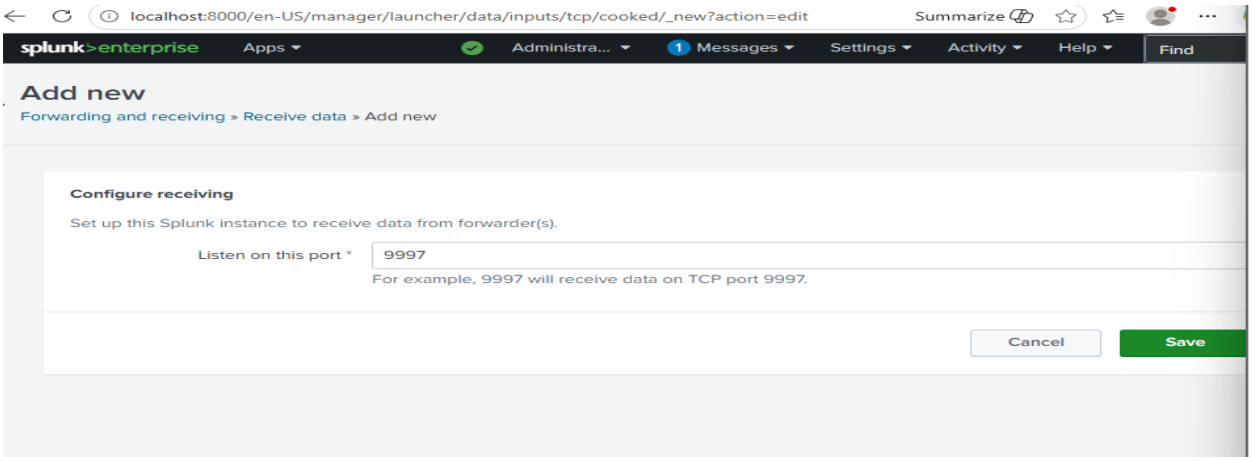


Splunk Configuration

Opening Splunk Receiver Port

Purpose:
Configure Splunk to receive forwarded logs.

Observation:
Port 9997 was enabled successfully for incoming data.



Splunk Universal Forwarder Setup on Debian

1 Downloading Splunk Forwarder

Purpose:
Download Splunk Universal Forwarder.

Observation:
The package was downloaded successfully.

```
debian@debian:~$ cd ~
wget -O splunkforwarder.tgz https://download.splunk.com/products/universalforwar
der/releases/10.2.3/linux/splunkforwarder-10.2.3-4d61cf8a5c0c-linux-amd64.tgz
--2026-05-18 13:34:16--  https://download.splunk.com/products/universalforwarder
/releases/10.2.3/linux/splunkforwarder-10.2.3-4d61cf8a5c0c-linux-amd64.tgz
Resolving download.splunk.com (download.splunk.com)... 108.159.102.18, 108.159.1
02.17, 108.159.102.26, ...
Connecting to download.splunk.com (download.splunk.com)|108.159.102.18|:443... c
onnected.
HTTP request sent, awaiting response... 200 OK
Length: 126163924 (120M) [binary/octet-stream]
Saving to: ‘splunkforwarder.tgz’
```

2 Installing the Forwarder

sudo mv splunkforwarder /opt/

Purpose:
Install Splunk Forwarder.

Observation:
The forwarder files were moved to the target directory successfully.

```
debian@debian:~$ sudo mv splunkforwarder /opt/
[sudo] password for debian:
debian@debian:~$ sudo /opt/splunkforwarder/bin/splunk start --accept-license
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R debian:debian /opt/splunkforwarder"

This appears to be your first time running this version of Splunk.

Splunk software must create an administrator account during startup. Otherwise,
you cannot log in.
Create credentials for the administrator account.
Characters do not appear on the screen when you type in credentials.

Please enter an administrator username: muslim
Password must contain at least:
```

3 Connecting Debian to Splunk

sudo /opt/splunkforwarder/bin/splunk add forward-server 192.168.1.34:9997

Purpose:
Connect Debian logs to Splunk.

Observation:
The forwarder connected successfully to the Splunk receiver.

```
debian@debian:~$ sudo /opt/splunkforwarder/bin/splunk add forward-server 192.168
.1.34:9997 -auth muslim:Kali01?1
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R debian:debian /opt/splunkforwarder"
Added forwarding to: 192.168.1.34:9997.
```

4 Monitoring Log Files

Purpose:
Add system and application logs to monitoring.

Observation:
Authentication, syslog, and Flask logs were added successfully.

```
debian@debian:~$ sudo whoami
root
debian@debian:~$ ls -l /var/log/auth.log
-rw-r----- 1 root adm 1548 May 18 13:55 /var/log/auth.log
debian@debian:~$ sudo usermod -aG adm debian
debian@debian:~$ sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/auth.
log -sourcetype linux_secure -index main
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R debian:debian /opt/splunkforwarder"
Added monitor of '/var/log/auth.log'.
debian@debian:~$ sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/syslo
g -sourcetype syslog -index main
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R debian:debian /opt/splunkforwarder"
Added monitor of '/var/log/syslog'.
debian@debian:~$ sudo /opt/splunkforwarder/bin/splunk add monitor /var/log/flask
_api.log -sourcetype flask_api_log -index main
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R debian:debian /opt/splunkforwarder"
Added monitor of '/var/log/flask_api.log'.
debian@debian:~$
```

5 Connection Test

nc -zv 192.168.1.34 9997

Purpose:
Verify connectivity and log forwarding.

Observation:
The test confirmed successful communication and log delivery.

```
debian@debian:~$ nc -zv 192.168.1.34 9997
192.168.1.34: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.1.34] 9997 (?) open
debian@debian:~$ logger "TEST: Debian logs are reaching Splunk"
```

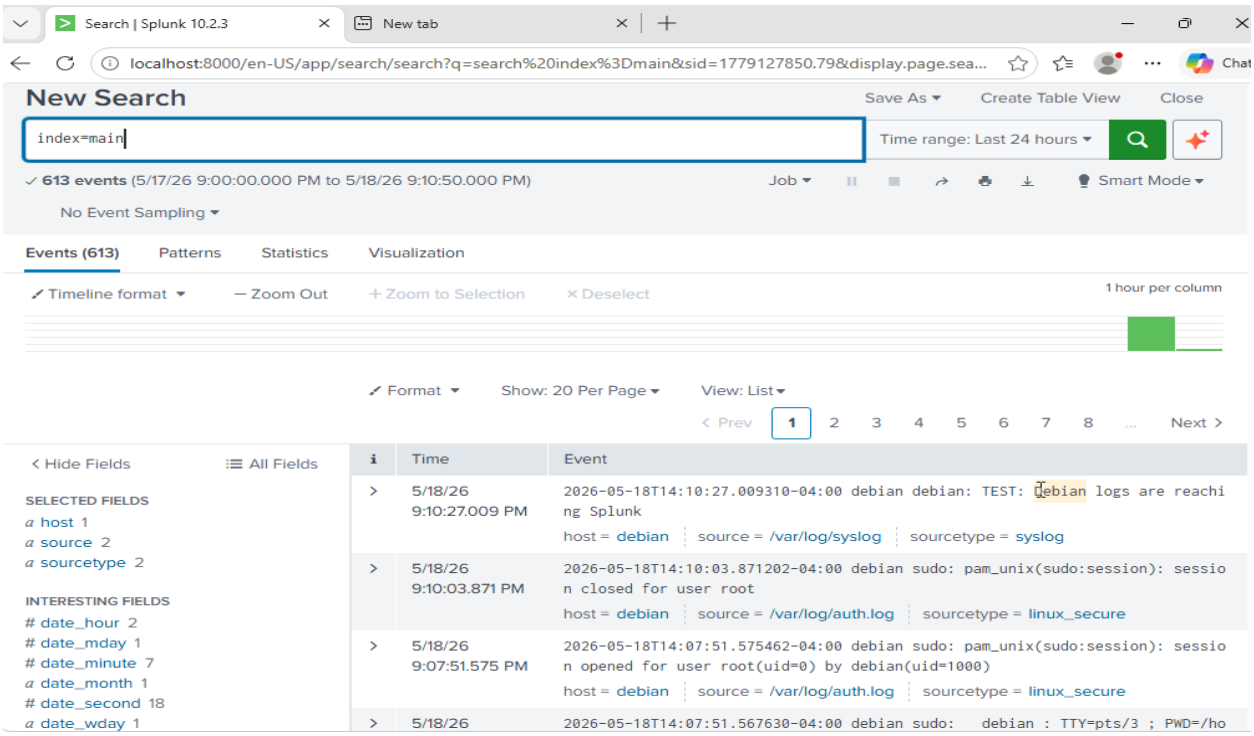
Splunk Log Verification

Main Index Verification

index=main

Purpose:
Verify that forwarded logs reached Splunk.

Observation:
The main index displayed incoming events successfully.


Search | Splunk 10.2.3
New tab
localhost:8000/en-US/app/search/search?q=search%20index%3Dmain&sid=1779127850.79&display.page.sea...
Chat
New Search
Save As
Create Table View
Close
index=main
Time range: Last 24 hours
613 events (5/17/26 9:00:00.000 PM to 5/18/26 9:10:50.000 PM)
Job
Smart Mode
No Event Sampling
Events (613)
Patterns
Statistics
Visualization
Timeline format
Zoom Out
Zoom to Selection
Deselect
1 hour per column
Format
Show: 20 Per Page
View: List
Prev
1
2
3
4
5
6
7
8
Next
Hide Fields
All Fields
SELECTED FIELDS
a host 1
a source 2
a sourcetype 2
INTERESTING FIELDS
date_hour 2
date_mday 1
date_minute 7
a date_month 1
date_second 18
a date_wday 1
Time
Event
5/18/26 9:10:27.009 PM
2026-05-18T14:10:27.009310-04:00 debian debian: TEST: debian logs are reaching Splunk
host = debian
source = /var/log/syslog
sourcetype = syslog
5/18/26 9:10:03.871 PM
2026-05-18T14:10:03.871202-04:00 debian sudo: pam_unix(sudo:session): session closed for user root
host = debian
source = /var/log/auth.log
sourcetype = linux_secure
5/18/26 9:07:51.575 PM
2026-05-18T14:07:51.575462-04:00 debian sudo: pam_unix(sudo:session): session opened for user root(uid=0) by debian(uid=1000)
host = debian
source = /var/log/auth.log
sourcetype = linux_secure
5/18/26
2026-05-18T14:07:51.567630-04:00 debian sudo: debian : TTY=pts/3 ; PWD=/ho

Splunk Monitoring & Detection

Splunk Enterprise was used to monitor the attack simulation in real time and collect logs forwarded from the Debian server and Flask API.

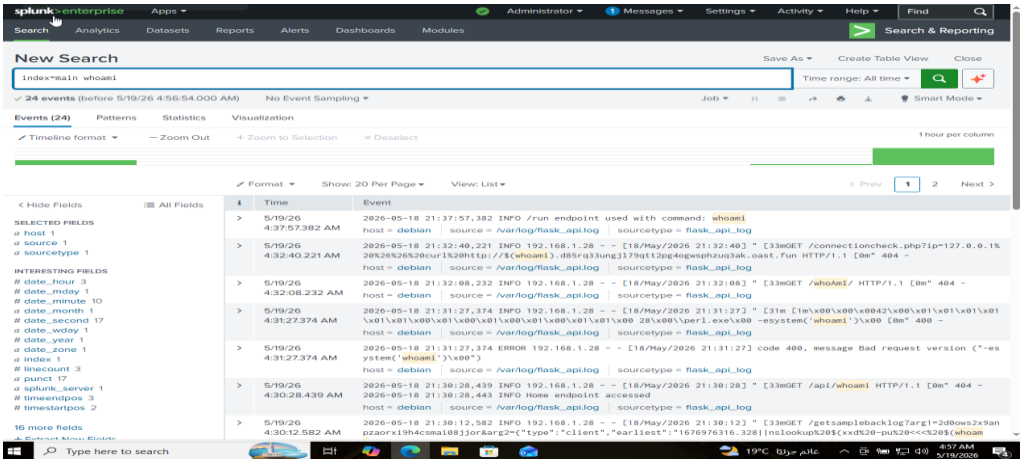
The **main index** was used to analyze authentication activity, API access requests, suspicious commands, and indicators of malicious behavior during the penetration testing phase.

User Enumeration – `whoami`

Search Query:`index=main whoami`

Purpose:
Used to detect command execution involving the `whoami` command.

Observation:
The logs confirmed user enumeration activity during the attack simulation.

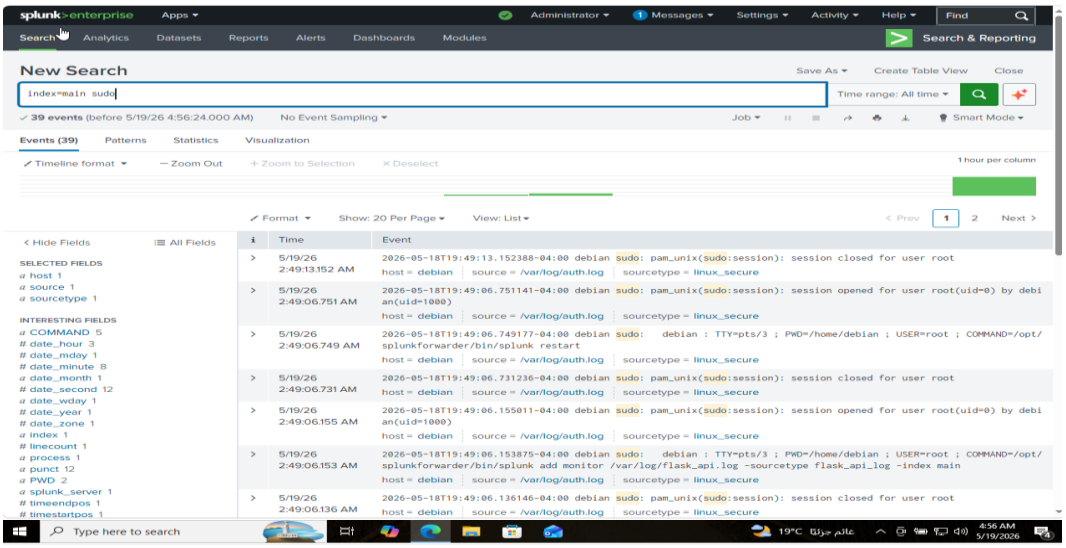


Privilege Escalation Monitoring – `sudo`

Search Query:`index=main sudo`

Purpose:
Used to identify commands executed with elevated privileges.

Observation:
Splunk detected sudo-related activity, indicating privilege escalation attempts.



API Endpoint Monitoring – GraphQL

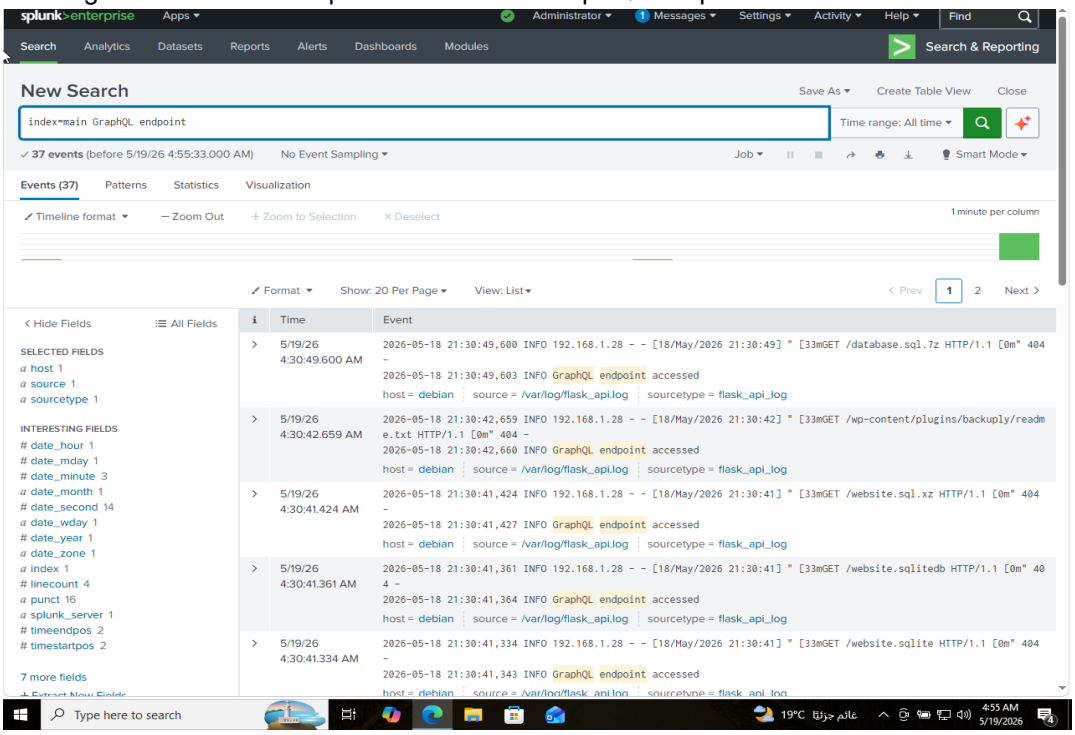
Search Query:`index=main graphql endpoint`

Purpose:

Monitored access to the GraphQL API endpoint.

Observation:

The logs showed active requests sent to the GraphQL endpoint.

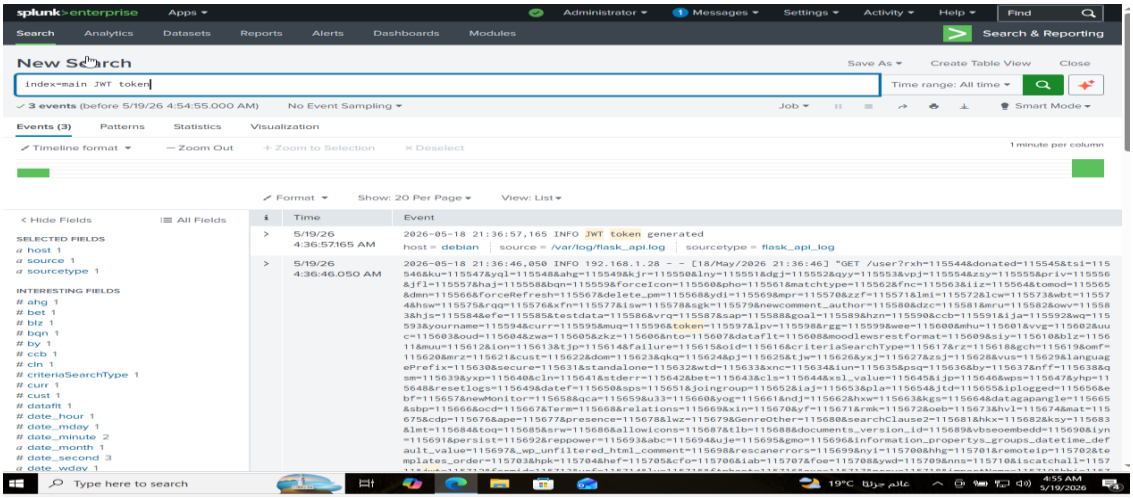


Token Activity – JWT

Search Query: `index=main jwt token`

Purpose:
Tracked JSON Web Token activity.

Observation:
JWT-related entries were recorded during API authentication.

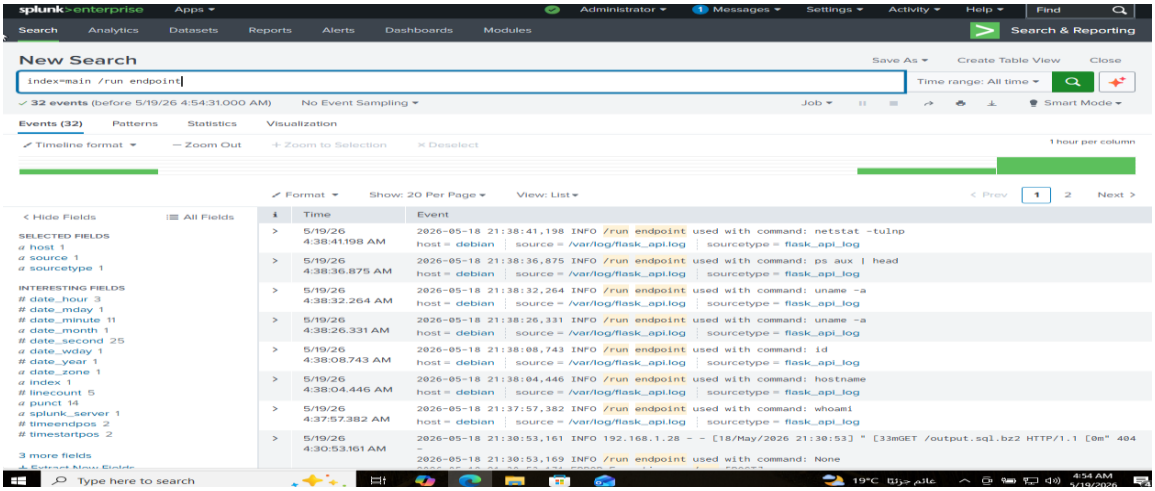


Runtime API Access – `/run`

Search Query: `index=main /run endpoint`

Purpose:
Used to monitor requests targeting the `/run` endpoint

.**Observation:**Splunk captured endpoint requests during the testing phase.

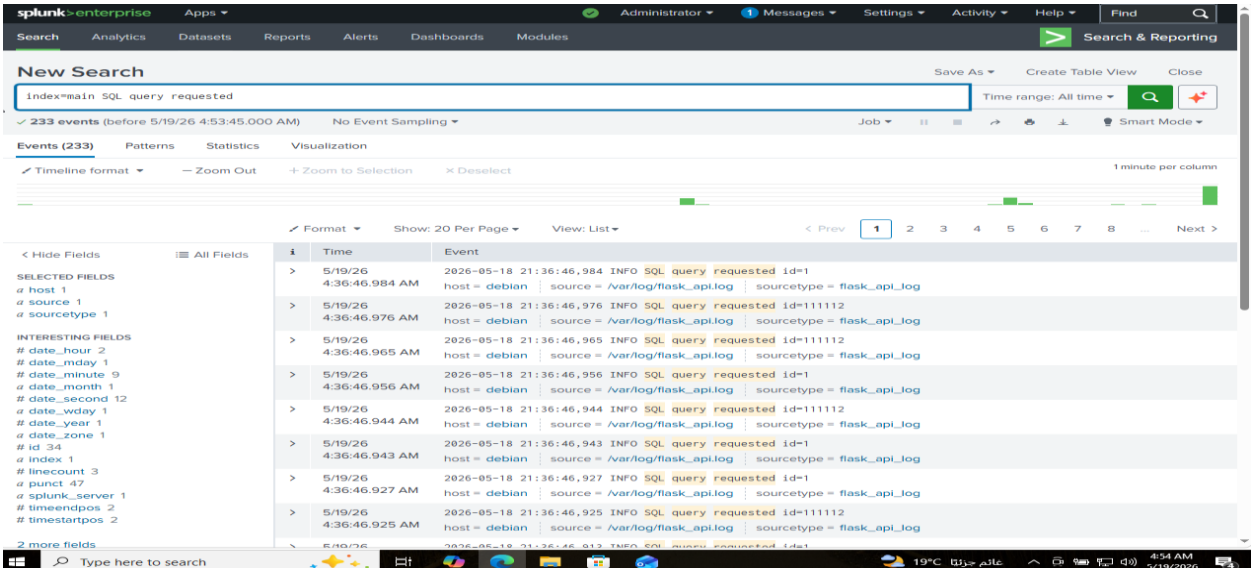


SQL Injection Indicators

Search Query:`index=main SQL query requested`

Purpose:
Tracked database-related requests.

Observation:
The logs captured SQL query activity against the application.

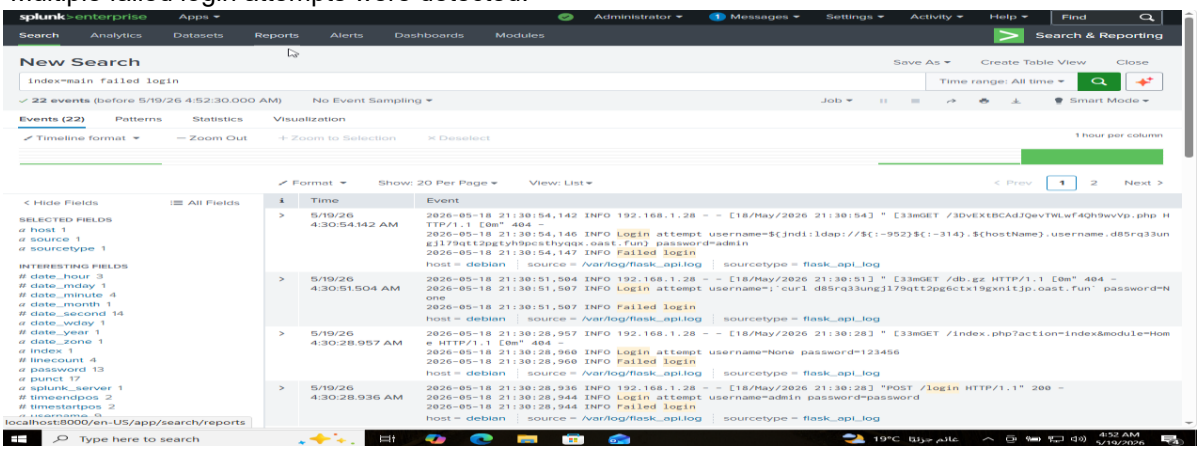


Authentication Failures

Search Query: `index=main failed login`

Purpose:
Monitored unsuccessful authentication attempts.

Observation:
Multiple failed login attempts were detected.

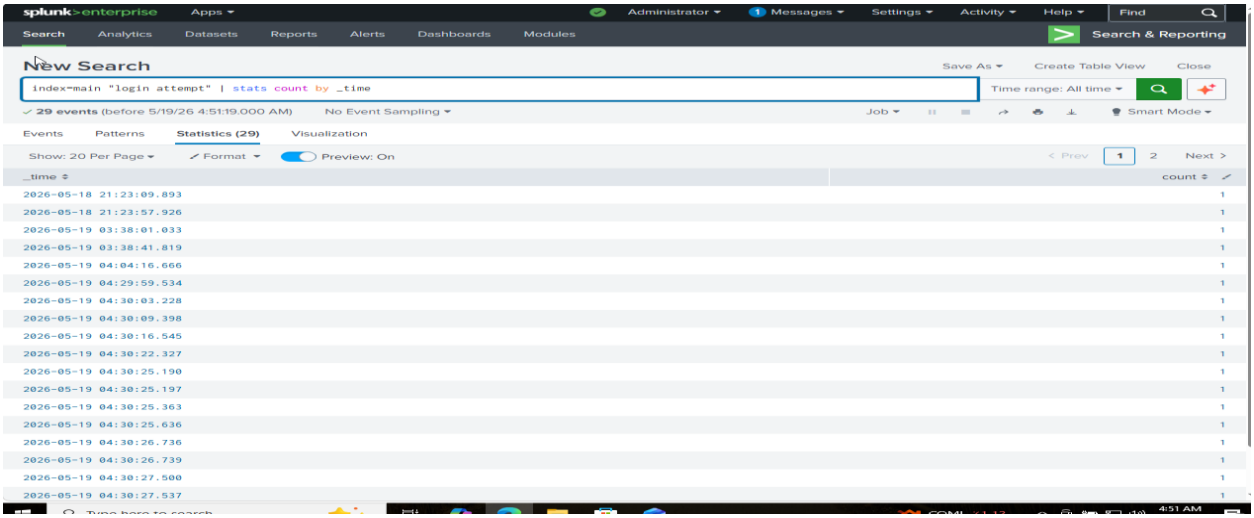


Login Attempt Statistics

Search Query:`index=main login attempt | stats count by _time`

Purpose:
Visualized login activity over time.

Observation:
The graph showed increased authentication activity during the attack window.

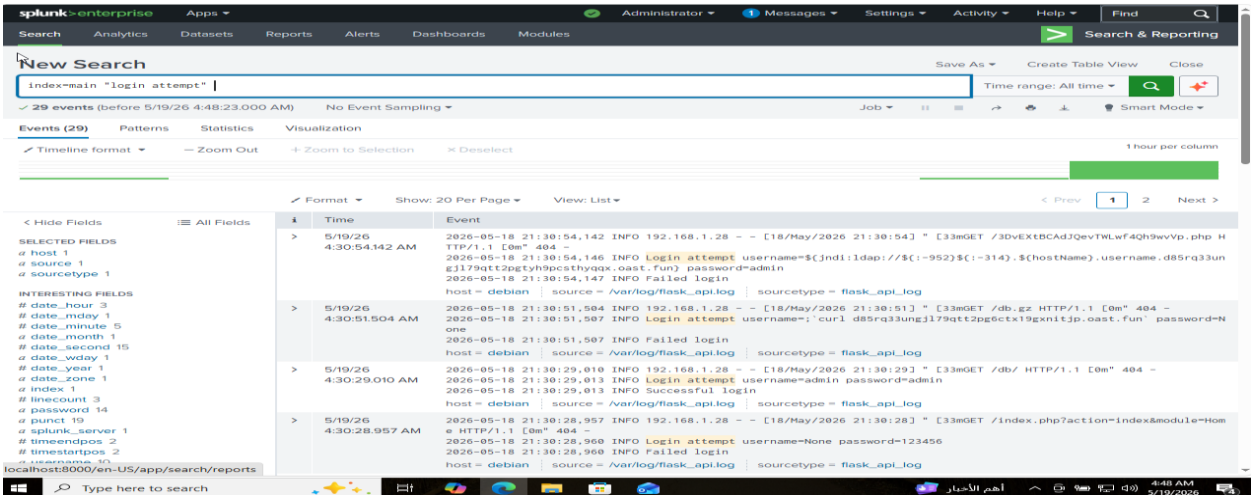


Login Attempts Review

Search Query:`index=main login attempt`

Purpose:
Reviewed all authentication-related events.

Observation:
The results provided a clear timeline of login activity.

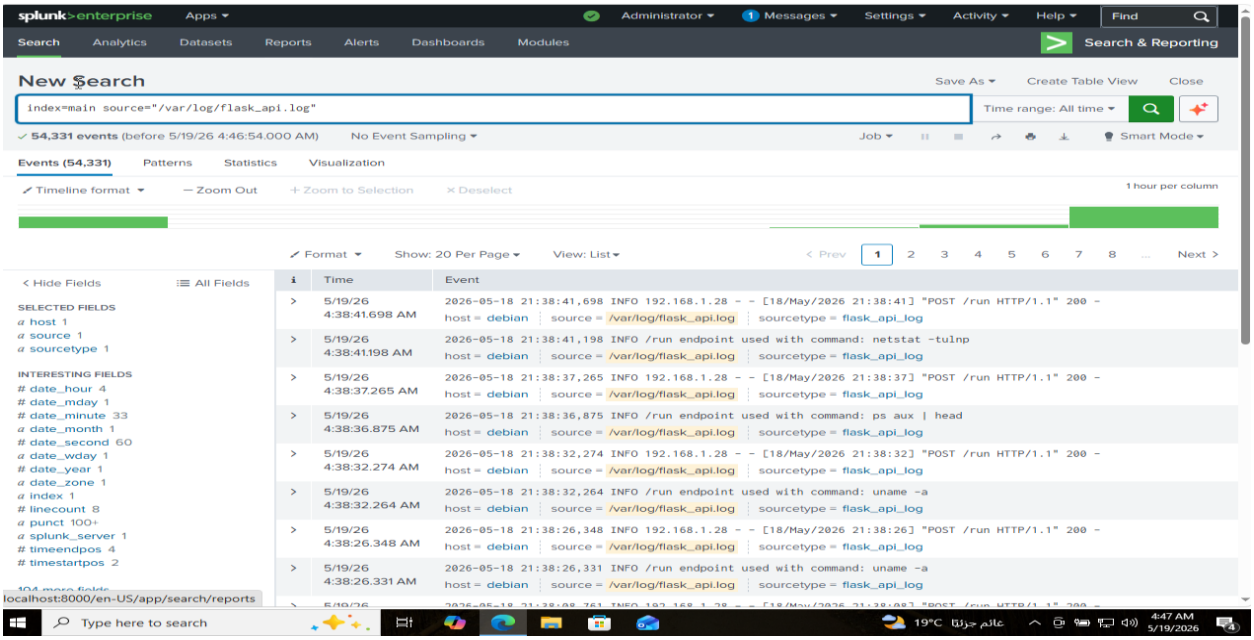


Flask API Log Source Verification

Search Query:`index=main source="/var/log/flask_api.log"`

Purpose:
Verified that Flask API logs were ingested into Splunk.

Observation:
The source was indexed successfully and logs were visible in real time.



Main Index Verification

Search Query: `index=main`

Purpose:
Used to verify that log data was successfully indexed in Splunk and available for investigation.

Observation:
The results confirmed that events were collected under the main index and were ready for analysis.

